



Power BI Developer Interview Guide & Quick Reference



Empowering Learners in Computer Science & Data Analytics



SCENARIO-BASED POWER BI DEVELOPER INTERVIEW QUESTIONS

1.Scenario: Data Modeling

Question:

You have sales data from multiple regions stored in different tables (e.g., Sales, Products, Customers). How would you design a data model in Power BI to enable efficient reporting?

Solution:

- Use a star schema design with a central fact table (Sales) and dimension tables (Products, Customers, Regions).
- Create relationships between the fact table and dimension tables using primary/foreign keys (e.g., ProductID, CustomerID).
- Ensure relationships are single-directional and avoid circular dependencies.
- Use DAX to create calculated columns or measures (e.g., Total Sales =SUM(Sales[Amount])).

Explanation: A star schema simplifies data modeling and improves query performance. Power BI's engine is optimized for this structure, enabling faster aggregations and filtering.



2.Scenario: Performance Optimization

Question:

Your Power BI report is slow. What steps would you take to optimize its performance?

Solution:

- Data Model Optimization:
 - Use star schema.
 - Avoid unnecessary columns and rows.
 - Use integer keys for relationships.
- DAX Optimization:
 - Avoid complex calculated columns; use measures instead.
 - Use `SUMMARIZE` or `ADDCOLUMNS` instead of nested `CALCULATE``.
- Data Source Optimization:
 - Use DirectQuery for large datasets or aggregate data at the source.
 - Enable query folding in Power Query.
- Report-Level Optimization:
 - Limit visuals on a single page.
 - Use filters to reduce data loaded into visuals.

Explanation:

Optimizing the data model, DAX, and report design ensures faster load times and better user experience.

3. Scenario: Incremental Data Refresh

Question:

You have a large dataset that updates daily. How would you implement incremental refresh in Power BI?

Solution:

- In Power Query, partition the data using a date column (e.g., `OrderDate`).
- In Power BI Desktop, enable Incremental Refresh in the dataset settings.
- Set parameters for `RangeStart` and `RangeEnd` to define the refresh window.
- Configure the incremental refresh policy (e.g., keep 2 years of historical data and refresh the last 7 days daily).

Explanation: Incremental refresh reduces the amount of data processed during each refresh, improving performance and reducing resource consumption.





4. Scenario: Row-Level Security (RLS)

Question: You need to restrict data access so that managers can only see sales data for their region. How would you implement this in Power BI?

Solution:

- Create a Role in Power BI Desktop (e.g., "RegionManager").
- Define a DAX filter for the role (e.g., `Sales[Region] = USERPRINCIPALNAME()`).
- Publish the report to Power BI Service and assign users to the role.
- Test the role using the "View As" feature in Power BI Desktop.

Explanation:

RLS ensures that users only see data relevant to their role, maintaining data security and compliance.

5. Scenario: Handling Missing Data

Question:

Your dataset has missing values in key columns (e.g., Sales Amount). How would you handle this in Power BI?

Solution:

- In Power Query, use `Replace Values` to fill missing values (e.g., replace nulls with 0).



- Alternatively, use DAX to handle missing values in calculations (e.g., ``IF(ISBLANK(Sales[Amount]), 0, Sales[Amount])``).
- Ensure data quality by validating the source data.

Explanation:

Handling missing data ensures accurate calculations and prevents errors in reports.

6. Scenario: Time Intelligence

Question:

You need to calculate Year-to-Date (YTD) sales. How would you implement this in Power BI?

Solution:

- Use DAX time intelligence functions:
DAX

YTD Sales = `TOTALYTD(SUM(Sales[Amount]), 'Date'[Date])`

- Ensure the `Date` table is marked as a date table and has a continuous date range.

Explanation:

Time intelligence functions simplify complex date-based calculations like YTD, MTD, and YoY comparisons.



7. Scenario: Dynamic Visuals

Question: You want to allow users to switch between different measures (e.g., Sales, Profit) in a visual. How would you implement this?

Solution:

- Create a measure selection table (e.g., `MeasureTable` with columns: MeasureID, MeasureName).
- Use a slicer for users to select the measure.
- Create a dynamic measure using `SWITCH`:
DAX Selected Measure =
SWITCH(SELECTEDVALUE(MeasureTable[MeasureID]), 1, SUM(Sales[Amount]), 2, SUM(Sales[Profit]))
- Use the `Selected Measure` in your visual.

Explanation:

Dynamic visuals enhance user interactivity and flexibility in reports.



8. Scenario: Data Integration

Question:

You need to combine data from an Excel file and a SQL database. How would you do this in Power BI?

Solution:

- Use Power Query to connect to both data sources.
- Transform and clean the data (e.g., remove duplicates, handle nulls).
- Merge or append the tables as needed.
- Load the combined data into Power BI for modeling and reporting.

Explanation:

Power Query enables seamless integration of data from multiple sources, ensuring a unified dataset for analysis.

9. Scenario: Custom Visuals

Question:

Your DAX measure is returning errors due to division by zero. How would you handle this?



Solution:

- Use Power BI Developer Tools to create custom visuals using D3.js or R/Python scripts.
- Alternatively, import custom visuals from AppSource if available.
- Test the custom visual thoroughly to ensure compatibility and performance.

Explanation:

Custom visuals allow for unique and tailored data representations, enhancing the report's effectiveness.

10. Scenario: Error Handling

Question:

Your DAX measure is returning errors due to division by zero. How would you handle this?

Solution:

- Use the `DIVIDE` function, which automatically handles division by zero:

DAX

Profit Margin = DIVIDE(Sales[Profit], Sales[Revenue], 0)

- Alternatively, use `IF` to check for zero:

DAX

Profit Margin = IF(Sales[Revenue] = 0, 0, Sales[Profit] / Sales[Revenue])

**Explanation:**

Proper error handling ensures that calculations are robust and prevent report failures.

11.Scenario: Hierarchical Data

Question:

You have a dataset with hierarchical data (e.g., Category > Subcategory > Product). How would you create a drill-down report in Power BI?

Solution:

- Create a hierarchy in the data model (e.g., Category > Subcategory > Product).
- Use a matrix visual and enable the hierarchy for drill-down.
- Optionally, use DAX to create calculated columns or measures for hierarchical calculations (e.g., `PATH` function for parent-child hierarchies).

Explanation:

Hierarchies and drill-downs allow users to explore data at different levels of granularity, enhancing data analysis.



12. Scenario: Dynamic Titles

Question:

You want to create a dynamic title for a visual that changes based on user selection (e.g., "Sales for [Selected Region]"). How would you implement this?

Solution:

- Create a measure for the dynamic title:

DAX

Dynamic Title = "Sales for " &
SELECTEDVALUE(Regions[RegionName], "All Regions")

- Add a text box or card visual and bind it to the `Dynamic Title` measure.

Explanation:

Dynamic titles improve user experience by providing context-specific information.

13. Scenario: What-If Analysis

Question:

You need to create a what-if scenario where users can adjust a parameter (e.g., discount rate) and see its impact on sales. How would you implement this?

**Solution:**

- Create a What-If Parameter in Power BI (e.g., Discount Rate).

- Use the parameter in a DAX measure:

DAX

Adjusted Sales = SUM(Sales[Amount]) (1 – DiscountRate[Discount Rate])

- Add a slicer for the parameter to allow user interaction.

Explanation: What-If analysis enables users to simulate scenarios and understand the impact of changes.

14.Scenario: Aggregations

Question:

You have a large dataset (e.g., 100M rows) and need to improve report performance. How would you use aggregations in Power BI?

Solution:

- Create an aggregation table in Power BI (e.g., pre-aggregate data at the year/month level).

- Set up the aggregation table in the data model and configure it to replace the detailed table for specific queries.

- Use DirectQuery for detailed data and Import Mode for aggregated data.



Explanation:

Aggregations reduce the amount of data processed, improving performance for large datasets.

15.Scenario: Custom Date Tables

Question:

You need to create a custom date table with fiscal year logic (e.g., fiscal year starts in April).
How would you implement this?

Solution:

- Create a custom date table using DAX:

DAX

DateTable =

CALENDAR(DATE(2020, 4, 1), DATE(2025, 3, 31))

- Add columns for fiscal year, quarter, and month:

DAX

Fiscal Year = YEAR('DateTable'[Date]) +
IF(MONTH('DateTable'[Date]) >= 4, 1, 0)

- Mark the table as a date table in Power BI.

Explanation:

Custom date tables are essential for handling non-standard fiscal calendars.



16.Scenario: Many-to-Many Relationships

Question:

You have a many-to-many relationship between two tables (e.g., Sales and Promotions).

How would you handle this in Power BI?

Solution:

- Use a bridge table to resolve the many-to-many relationship (e.g., a table linking Sales and Promotions).
- Create relationships between the bridge table and the main tables.
- Use DAX to handle calculations (e.g., `CALCULATE` with `USERELATIONSHIP`).

Explanation:

Bridge tables simplify many-to-many relationships, ensuring accurate data modeling.

17.Scenario: Advanced DAX - Ranking

Question:

You need to rank products by sales within each category. How would you implement this?

**Solution:**

- Use the `RANKX` function in DAX:

DAX

Product Rank =

RANKX(

FILTER(Products, Products[Category]

= EARLIER(Products[Category])),

SUM(Sales[Amount])

)

Explanation:

`RANKX` allows for dynamic ranking based on specified criteria, enabling advanced analysis.

18. Scenario: Handling Slowly Changing Dimensions (SCD)**Question:**

You have a slowly changing dimension (e.g., Customer addresses). How would you handle this in Power BI?

Solution:

- Use Type 2 SCD logic in Power Query or SQL to track historical changes.
- Create a table with start and end dates for each record.
- Use DAX to filter data based on the effective date:

DAX



```
Active Customers =  
CALCULATE(  
    DISTINCTCOUNT(Customers[CustomerID]),  
    Customers[StartDate] <= TODAY() && Customers[EndDate]  
    >= TODAY()  
)
```

Explanation: SCD Type 2 ensures historical data accuracy and supports time-based analysis.

19. Scenario: Advanced Visuals - Waterfall Chart

Question: You need to create a waterfall chart to show the contribution of different factors to a total (e.g., profit breakdown). How would you implement this?

Solution:

- Use the built-in Waterfall Chart visual in Power BI.
- Map the data fields (e.g., Category, Value).
- Use DAX to calculate intermediate values if needed:

DAX

```
Profit Contribution =  
SUM(Sales[Profit]) - SUM(Sales[Cost])
```

Explanation:

Waterfall charts are ideal for visualizing incremental contributions to a total.

20. Scenario: Dataflows vs. Datasets

Question:

You need to decide between using Dataflows and Datasets in Power BI. What factors would you consider?

Solution:

- Dataflows:
 - Use for reusable data preparation (e.g., cleaning, transformation).
 - Ideal for centralized data management.
- Datasets:
 - Use for modeling and reporting.
 - Ideal for specific reports or dashboards.
 - Consider factors like data reuse, performance, and team collaboration.

Explanation:

Choosing the right tool depends on the use case, data complexity, and organizational needs.

